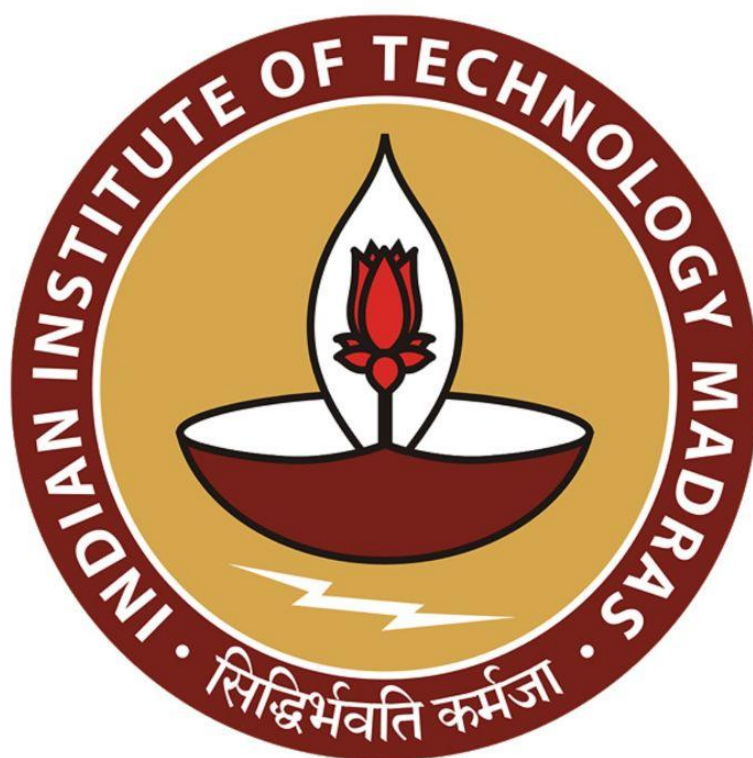
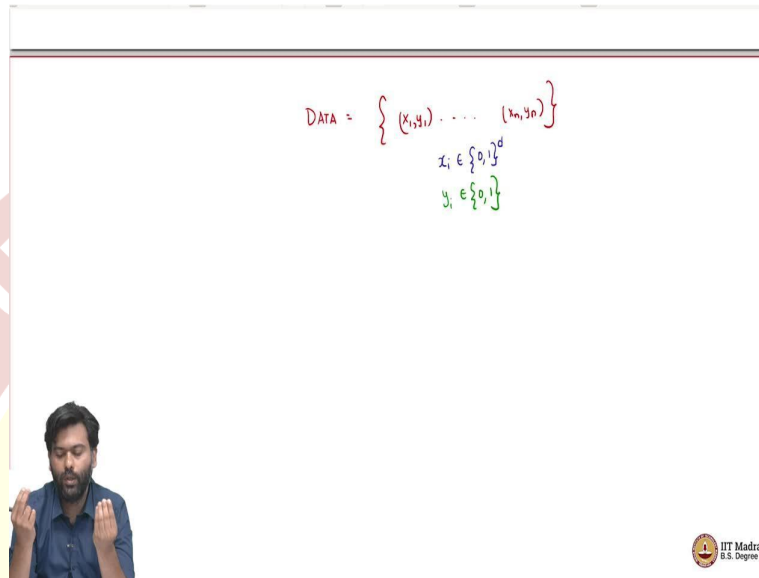




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science & Engineering
Indian Institute of Technology Madras
Generative Model-Based Algorithm

(Refer Slide Time: 0:13)



Now what we are going to see is a generative model based algorithm, so for to set things up. So let us say we have our data as usual as $(x_1, y_1), \dots, (x_n, y_n)$, this is a binary classification problem so the labels y_i 's are either 0 or 1, as usual, nothing changes there. But in this particular example, that we are going to deal with, we are going to assume make a small change from our usual assumption, we are going to assume that our features are d dimensional, but then they are binary. That is, you have d features, but then each feature can take the value either a 0 or a 1. So that is the only possibilities that we are going to allow for the features.

So we will see this as prototypical example for a generative model based algorithm that will come out of it. But before we go into the algorithm and our assumptions and so on, let us ask the question what does it mean, I mean, is there a reasonable example where the features are binary? So can the features be a binary in a binary classification example? Of course, it can be binary, but then in a practical, useful setup, can you think of an example application where the features could be binary? Pause and think about this, I will tell you the answer 1 example now.

(Refer Slide Time: 01:48)

Handwritten slide content for SPAM CLASSIFICATION example:

- $y_i \in \{0, 1\}$
- EXAMPLE: SPAM CLASSIFICATION
- $x_i \in \{0, 1\}^d$ (with a red arrow pointing to d and the text "# Words in dictionary")
- Eg: "Hello, how are you?"
- [0 0 0 ... 1 1 0]
- Words: None ARE HELLO HOW YOU ZEDRA

The slide also features a small video feed of a man in the bottom left corner and the IIT Madras B.S. Degree logo in the bottom right corner.

Here is an example. One example that we will see is the problem of spam classification. So, what is the problem of spam classification? Well, you are given a dataset where you have a bunch of let's say emails, and some emails are tagged as spam, some emails are tagged as not spam, we can say that the spam means the class is 1 and non-spam means the class is 0, so the labels are done with it.

Now emails could be written in different lengths, could have different words, all sorts of things it is an unstructured data. So now we need to somehow convert this into, as fixed size features, that is you need to convert your email into a feature vector, only then we can work with our algorithms, because in our algorithms, you are always going to assume so far, and we will continue to that our features are all d dimensional for some d , d could be large, but then it has some fixed value d . So which means that because the emails could be of different lengths, we need to somehow convert each email into a fixed length feature.

Now what might be a good way to convert emails into fixed length features? Think about this. I will give you one example, one way to do this now. One way to do this would be we have x_i , which is an email, which becomes a vector in d dimension, where d is the number of words in the dictionary, in the standard dictionary, let us say, what does this mean? This means that, let us say my email for an example, I take an email, which reads as follows. Hello, how are you? This is my email. So now what I am going to do is, I am going to convert this email into a d dimensional vector, which has, lots of 0's and a couple of 1's.

So now, what is this? All everything else is 0, except where I have indicated 1, everything else is 0. Now, what is this feature correspond to? Well, it corresponds to the dictionary, length of the dictionary, which means that maybe here is a word which starts in I am just, I do not know what the first word in the dictionary is, but let us say this starts this has to start in a, maybe I am just writing about as the first word. Maybe here is a word which is ARE, maybe here is a word, which is hello, maybe this word is how, maybe this word is you, maybe this word is zebra.

So every word in the dictionary gets one component of this feature vector. So if there are 10,000 words in your dictionary, then you have a 10,000 dimensional feature vector. But now to map our specific email which says hello, how are you to this 10,000 dimensional feature vector, we are going to do the following, we are going to take every word in our email, and then mark that corresponding entry in this feature vector as 1. So for example, hello is a word in the email, so the hello component gets a value 1, how is in email, is in the email how component gets 1, same for are and you. However, zebra is not in this email, so the zebra component gets 0. In fact, any other word is not in this email, so all the other components become 0.

So this is how I can convert my email into a fixed length encoding of the dimension equal to the length of the dictionary. And by no means I am saying that this is the way to do this the only way to do I am just giving you one way to do this. So feature, you know, how to create features is depends completely on a lot of other things also in some sense, what do you think are meaningful things that might be relevant for the problem at hand becomes important, this is a usual basic thing that one can do where you can take the email and then convert it into a 0, 1 representation. There are other sophisticated ways to do this, but then for our purposes, for this algorithm, this is a simple way that is good for explaining this algorithm. And this does not do that badly in practice, as we will see later on, as well, so it is a reasonable assumption to make.

(Refer Slide Time: 06:44)

$x_i \in \{0,1\}$

Eg: "Hello, how are you?"

→

Not	ARE	HELLO	HOW	YOU	ZERO
0	0	0	1	1	0

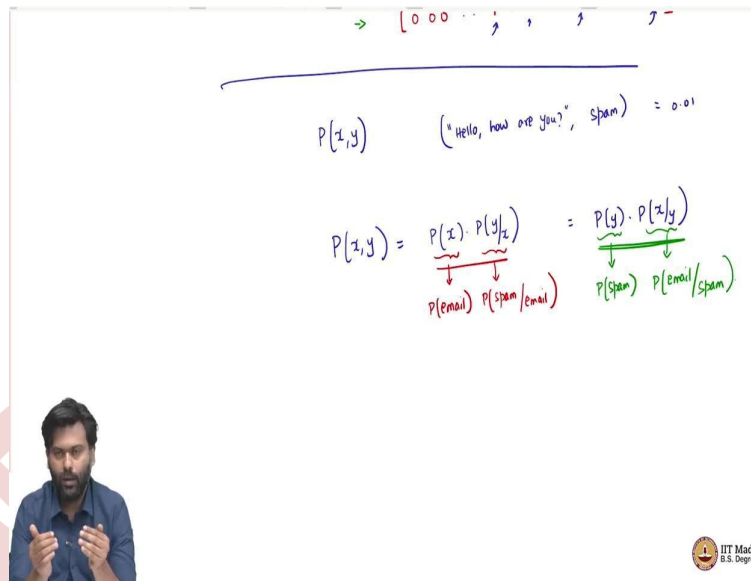
$P(x,y)$ ("Hello, how are you?", spam) = 0.01

IIT Madras
B.S. Degree

Okay. Now, okay so now what we have said is that here is a problem binary classification problem where the features are binary, the labels are 0 or 1. So now remember we wanted to do a generative modeling assumption, which means that we want to model we want to model $P(x,y)$. In this case, in this context, what does that mean? That means that we want to associate probabilities with every email, label pair. For example, for this particular email, hello, how are you? Now if I tell you, this is the email, so hello, how are you oops, how are you, spam, let us say if I give this pair, then this pair should get a probability so that this pair could have been generated from the underlying process that generates the data, maybe there is a probability of 0.01 for this perhaps.

So similarly, there will be a probability for Hello, how are you and non-spam, which means that for every email possible, and every label possible this pair we need to associate a probability. Now we need to associate some way, so we need to come up with some way to associate these probabilities. Now either we can see that well, there is some assumption that I am going to make on the structure of this joint distribution of features and labels, that is emails and labels or I can do the following.

(Refer Slide Time: 08:11)



The slide shows a handwritten equation for the joint probability $P(x, y)$. At the top, it says $P(x, y)$ for $(\text{"Hello, how are you?"}, \text{spam}) = 0.01$. Below this, the equation is written as $P(x, y) = \underbrace{P(x)}_{P(\text{email})} \cdot \underbrace{P(y|x)}_{P(\text{spam}|\text{email})} = \underbrace{P(y)}_{P(\text{spam})} \cdot \underbrace{P(x|y)}_{P(\text{email}|\text{spam})}$. A small video inset of a man speaking is in the bottom left, and the IIT Madras logo is in the bottom right.

I can use the fact that $P(x, y)$ can be written in 2 different ways, so one way is I can write this as $P(x)$ into $P(y|x)$, this is one way to write this, what is the other way? Well, we know the other way, the other way is $P(y)$ into $P(x|y)$. So what does this mean? this What does I mean, of course, this is just a formula that comes from probability, but in our context, what does this mean? This means that, well, if I am going this route, that I am going to say, I am going to model $P(x, y)$ by separately modeling $P(x)$ and $P(y|x)$, then it means that I am saying, but I will model the structure of how the email is generated, which is to say that, will this is the probability of email and I will model what is the probability of spam or non-spam given email.

I am going to make some assumptions about how the email is generated and now given the email how the label is generated, I can make these 2 assumptions I can separately model $P(x)$ and separately model $P(y|x)$. Now, if you think about this, at the end of the day, what do we care about? We care about $P(y|x)$. So meaning given a new data point, we want to make a prediction as to whether this data point, this email is a spam email or a non-spam email, which means that if I plug in $P(y)$ in the probability equation $P(y|x_{\text{test}})$, and if this probability for spam is 0.9 then I would predict it as spam. If the probability is 0.05 then I would predict it as not-spam. So I am going to look at these probabilities and make my decision later.

So if I am going the first way, where I am modeling both P of email, and $P(y)$ given email, then essentially it means that I know the essential structure of how y is going to be given x , so because I am modeling $P(y|x)$. And I only need $P(y|x)$ any way to make my prediction, so at

the end of the day, I am going to make a prediction and for the prediction purposes, I only need $P(y|x)$.

Well, if I know how $P(y|x)$ is already, then I do not perhaps I will not gain that much by modeling $P(x)$ also, separately, because I know essentially what I am saying, if I know $P(x)$ and $P(y|x)$, structurally, then it means that I am saying that I know how the label is generated given the feature, which is I know the structure behind $P(y|x)$, that is all I need anyway. So if I know that structure, if I am making an assumption on that structure, it is good enough, I do not necessarily have to make another assumption on $P(x)$. So in some sense, I might as well do a discriminative modeling where I will just model $P(y|x)$, instead of modeling $P(y|x)$ and $P(x)$.

On the other hand, if you are going this route, well, here there is no structure that I am assuming about how the y is generated given the x , instead I am assuming how the labels are generated, which is $P(y)$. So this is P of let us just think of this as probability of spam and this guy as probability of email given spam. Here, I am saying, well I am going to make an assumption about how spam emails look like that is given spam how does what is the chance that a particular email comes about, and given not spam what is the probability that a particular email is.

So this is a more meaningful assumption in some sense, because I might have some way of understanding for every class which is the spam or the not spam class, how does the distribution of emails look like which means that I can perhaps model $P(x|y)$ and model what is on an average how many spams are going to be there are many non-spam are going to be there. So what is the likelihood that I see your spam or non-spam which is being modeled $P(y)$. So this is a better way to model $P(x,y)$ and this is what we are going to do for this particular example, we are going to go the way of modeling $P(y)$ and $P(x|y)$. So now we will see how to do this modeling exactly now.

(Refer Slide Time: 12:49)

GENERATIVE STORY

STEP 1: DECIDE the Label by tossing a coin
 $P(y_i = 1) = P$

STEP 2: DECIDE Features using the label in step 1 by $P(x_i | y_i)$

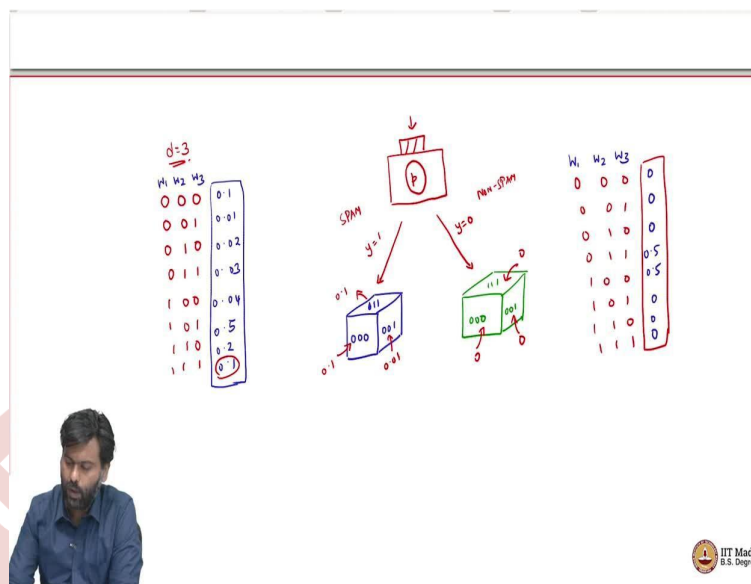
IIT Madras
B.S. Degree

Okay. So as usual, when you are doing a generative model, what you would do is you will assume a generative story. Now what is the story, we are going to assume that generates our dataset so that is what we need to decide. So in this particular generative story, and this is one story that this example is going to this algorithm is going to make assumption on. Again, you can make your own generative story, but then this is one story we will start looking at, and we will see if it is a good story or not as we go along.

So this story has 2 steps. Step 1, decide the label by tossing a coin that is we want to generate the i -th data point in the training set and I am assuming that the probability that the i -th data point corresponds to the label 1, which is let us say spam, is some P . So basically, I have a coin with probability P , I am tossing it if it falls heads, then I am going to say, well this data point i -th data point is a spam email. I have not told you what the email is set, but I have told you what the label is, what type of email I am trying to generate that is step 1.

Now step 2 is to generate the email itself. Now step 2. Step 2 is basically decide the features so you have decided the label, now you have to decide features using the label in step 1 by $P(x_i | y_i)$. That is, you are going to make some assumptions on $P(x_i | y_i)$ and we will see what this assumption is, which means that if the coin fell heads, then I am going to draw an email from a distribution over emails, given that the email is a spam email. So we will see what this means, this distribution means now.

(Refer Slide Time: 15:17)



So let us make this more precise, let us look at our, the standard way of that we have been thinking about these things. Basically, in pictures, it looks like the following, you have a box, the box has a coin inside it and I am pressing this coin, this coin will tell me what the label is. So the label can be either 1, or the label can be 0, that is, it can be heads, this can be tails. Now, let us say it is heads. So if it is heads, then we need to generate a spam email, let us say. Now how, which means we need to generate features. Now how many dimensions are there in this features? There are d dimensions, d words in the dictionary, so there are d dimensions.

Now if let us say there is there are only d is 3. So which means that the set of all possible feature vectors with d equals 3 or what. How many feature vectors are there? So there are only think about this, I am going to tell you the answer now, pause and think about this. So if d is 3, then there are only 8 different feature vectors possible, which are as follows, these 8 bit patterns are the only set of features that you can get. So when these 3, it means that there are only 3 words in my dictionary, so which is a simplified assumption, I mean, in practice, you are going to have these 10,000, but just to illustrate, I am saying if d is 3 take this as an example, there are only 3 words.

Now, every email well, the first one that is an empty email, it does not have any of these words, or an email can have only word 3, or an email can have any subset of these words. So you have 8 possibilities. So every word can either occur or not occur in the email. So there are 2 possibilities for every word, and there are 3 words each word can occur or not occur. So it is $2 \cdot 2 \cdot 2$, there are 8 possibilities. So now, I need to pick one of these emails. So if it is

spam, so each of these emails can be associated to a spam or not spam, we do not know. So let me draw the same thing on this side also. So for the non-spam case, it is the same email 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0 and so on, we have this.

Now imagine it as follows. I toss the coin, the coin fell heads, which means that I need to generate a spam email. So this is the spam world, this is the non-spam world. Now I need to pick one of these guys, so one of the 8 emails. Now imagine as if there is an imaginary 8 faced dice, where for each of these emails, the dice tells you what the probability that this face shows up. So maybe this value is 0.1, this is 0.01, this is 0.02, 0.03 and this making these numbers up 0.04, maybe this is 0.5, 0.2, 0.1.

So maybe there is this dice. So this imaginary 8 faced dice, of course, I am standard dice has only 6 phase, but imagine that this has 8 phase. Now this is the phase corresponding to 0, 0, 0, this phase corresponds to 0, 0, 1, 0, 1, 1, 1, and so on, so there are 8 phases in this dice. Now what I am going to do is, all these phases are not equally likely, in a standard dice, all phases are equally likely here, there is some probability associated with each of these phases. So these probabilities, we do not know. But then let us say as an algorithm, we do not know, but then we are assuming that there is some set of probabilities.

So I am going to toss this, sorry, I am going to roll this dice, the dice will fall in some phase, that phase will have a binary pattern and that pattern is corresponding to some email, which is what I will generate. So what does this mean? So now, all I have done is I have tossed the coin, the coin fell heads, which means that I need to generate a spam email. So I went into the spam room, and the spam room had a dice, 8 faced dice, I rolled the dice, the dice showed me what the feature is.

Now generate the second point, second data point in my data set, I again, flip the coin. Now this time, let us say it fell tails, which means that I need to generate a non-spam email, which means I go into the non-spam room. Well, it is the same set of emails that are possible that you can generate, but maybe the non-spam email has a lot of 0's, so me maybe these words, combination cannot occur in non-spam email, let us say, and this is 0.5 and 0.5. It is the same set of possibilities, but then the probabilities are different.

Now, which means that there is, again, an 8 faced dice, but then the chance of each of these phases showing up is different here. So it is the same 8 faced means the dice is still 8 faced, but then the probabilities are different. So I am going to indicate that using a different color

here. Let us say this is a different dice, so this the faces are same, but then now for example, this 1, 1, 1 had probability 0.1, this which is this, 0, 0, 0 also had 0.1, 0, 0, 1 had 0.01. On the other hand, this guy has 0, this guy has 0, in this particular case, this guy has 0 also, maybe different other faces. In this case 0, 1, 1, and 1, 0, 0, are the ones that have 0.5 and 0.5.

Now these, these numbers could be anything, we do not know this, we are just assuming that there is there is a set of such numbers, which means that all we are saying is that you have a coin to decide which is the label and you have 2 dice, imaginary 8 faced dice, one for the spam world, one for the non-spam world. Why does this make sense? This makes sense because let us say you have an email, which has the words, lottery, money, reply immediately, and whatever this dollar and things like this.

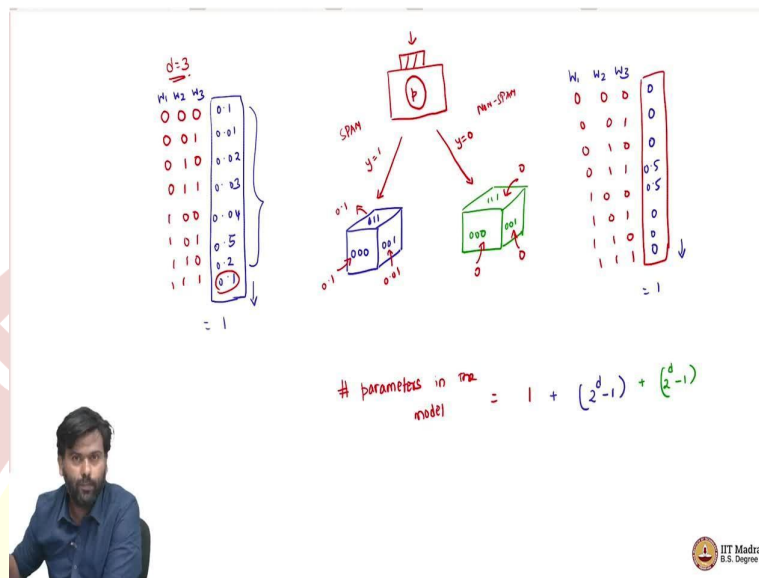
So if I see these words, now this email is more likely to be a spam email so that which means that it is the same email, which means that the encoding for this email is the same, both in the spam world and the non-spam world, but then the chance that, it is comes from a spam dice is higher, whereas the chance that it comes from a non-spam dice is lower, you cannot immediately say that the non-spam world cannot have such an email, maybe you are a lottery business person for whom this email still makes sense.

So the chance that this would have come from a spam email will be higher, which means that, that face will have higher probability on the blue dice, whereas the same face will have lower probability in the green dice. So the faces are the same, but then the chance that this face will show up when you roll the die is different for different spam and non-spam. Hope that is clear. So now let us think about, we have put down a model, so now we are seeing that every data point in my training set is generated like this, I go through this 2 step process, first I toss the coin, get the spam or non-spam, then I roll the corresponding dice and get the features, I do the same thing n different times I get n data points.

Now the question that I want to ask is, well, we have put down a model, so which means implicitly we are saying what is $P(y)$, and what is $P(x|y)$, that is what we have said, which means essentially, we are saying $P(x,y)$, because once I know $P(y)$ and $P(x|y)$, I can multiply these 2 things to get $P(x,y)$. And for every email, label pair, I now have a probability associated with it, which comes from this assumption of their existing coin and 2 dice. Now if I have to learn, I have not put on an algorithm. But if I have to learn from this data, I need to know how many parameters determine my entire distribution, probability distribution. In this

case, the parameters are determined by these coins and this dice. So let us see how many parameters are there in this model?

(Refer Slide Time: 23:44)



So the number of parameters in the model. Can you think about how many parameters are there in this model? You are already seeing it just have to think about and make sure you get the number, pause and think about this. I will tell you the answer now. Well, the number of parameters is there is one parameter to decide whether the label is 1 not 0, spam or not spam, that is just a coin tosses bias P . So now, for this spam world there is a dice, but how many faces does this dice have? This dice has, if there is d version in my dictionary, this dice has 2^d faces. If there are 3 words that I had 8 faces, if there are 4 the dice will have 16 faces, if there are 16 binary digits of length 4, if there are d words, then this dice will have 2^d faces.

Now, each of these face is giving the probability of that face showing up so the sum of the probabilities are going to be 1. So if I sum these guys up it will be 1, if I sum these guys up it will be 1 so that should happen. Because the dice should fall on I want at least I mean exactly 1 of these faces, so these are mutually exclusive events, each of these face showing up and then if I sum up the probability that should be 1.

Now, there are 2^d faces and then the sum of probabilities is 1, which means that if I fix $2^d - 1$ numbers, for example, if I fix the first 7 numbers here, the 8th number, I can calculate by just doing 1 minus the sum of the 7 numbers. So essentially, the number of free parameters that I have here is for the blue dice is $2^d - 1$ and for the maybe I should put this guy in the

corresponding color 1 for the coin $2^d - 1$ for the blue dice, and $2^d - 1$ for the green dice, because they could be completely different numbers, so in this non-spam world.

So I need to know all these numbers. So only if I know all these numbers, these many numbers, then I have the ability to generate a new x, y parameter, which means I have completely understood this generating process. If you tell me these many numbers, then I have complete control over how the data is generated, which means I can do anything. For instance if I want to make predictions, I can calculate $P(y|x)$ for any x test data point x , so for which I need all these numbers. So which means that as an algorithm, if I am trying to understand, learn from data that means I need to learn these many numbers, so this is how many.

(Refer Slide Time: 26:39)

parameters in the model $= 1 + (2^d - 1) + (2^d - 1)$

$\uparrow$ to decide label $\uparrow$ $P(x|y=1)$ $\uparrow$ $P(x|y=0)$

$= 1 + 2(2^d - 1)$

$= 2^{d+1} - 1$

ISSUE

- Too many parameters!
- Not a reasonable story.

IIT Madras
B.S. Degree

So I need 1 number to decide the label. So this is $P(x|y)$ again, $P(x|y=1)$, and this guy is $P(x|y=0)$. So in total, I need, $1 + (2^d - 1)$, which is $2^{d+1} - 1$ parameters. Now the question is, is this a reasonable model? We have put down some generative model as to how the data is generated, now we have to ask ourselves, is this a good thing or a bad thing? Well, if you have to learn from this data from data, and if you are positing that this is the model that is generating the data, then you need to learn $2^{d+1} - 1$ parameters to understand the model itself.

But even for moderate sizes of d , so if you take a standard dictionary, you are going to at least have 10,000 words. And now, if you need $2^{d+1} - 1$ parameters, that means that you have to learn $2^{10,000}$ order of $2^{10,000}$ parameters, that simply too many. So $2^{10,000}$ is simply too many, even for modern GPU's to learn, so it is astronomical. So the issue with this model that we

put down is that there are too many parameters, which means though it is a model and this is the important point, though this is a model that generates the data and we are free to choose whatever model, whatever assumption we want to make about how the data is generated.

So now the question is, there are 2 things to think about whenever you are creating a generative model. One is, well is it a reasonable model to explain your data? That is the first thing. The second thing is that, is it reasonable enough that you can learn under this model? Both are important. Now this model may be a reasonable model, maybe this this is a model to explain our data reasonably well. But the problem is, it is not reasonable enough that we can learn from this, simply because there are too many parameters. What does that mean? That means that we need to change our generative story, so this story will not work in practice. This story is hard to learn from in practice, unless your d is very-very small, which in this spam classification case is clearly going to be large. And this story we have to give up on the story.

So now we need to change our story, so that is what this means. So now we need not a reasonable story. So now we need to come up with an alternate story that explains our data, which also has, meaningful number of parameters that we can we can learn potentially. Now what I am going to do next is put up another alternate story, which will be much more reasonable in terms of whether you can learn from it or not in other words the number of parameters that it will involve will be much lesser than the parameters that we have in this story, let us look at that alternate story.